■ Intelligent Resume Parser & Job Matcher

> An AI-powered HR tool that reads resumes, understands them deeply, matches them to job descriptions, and ranks the best candidates — automatically.

■ What Is This Project?

Imagine you are an HR manager at a mid-sized tech company. You post a job for a **"Senior Data Scientist"** and within two days you receive **400 resumes** in your inbox — PDFs of all shapes, sizes, formats, and languages (some even written in Hinglish, a mix of Hindi and English).

Reading all 400 manually would take weeks. And even then, human bias and fatigue lead to missed talent. This project solves that problem.

You upload all 400 resumes and one job description. The system reads every resume, understands what skills and experience each candidate has, compares them to what the job requires, and hands you back a ranked list — **Candidate #1 is your best match, Candidate #400 is the weakest.** All in under a minute.

■ Real-Life Example: Walking Through the Full Pipeline

Let's follow one resume through the entire system so you can see exactly what happens at each step.

The Resume

Priya Sharma applies for the role of **Senior Data Scientist**. Her resume PDF contains:

```
Priya Sharma | Delhi, India
Skills: Python, Machine Learning, TensorFlow, Data Visualization, SQL
Experience: 4 years at Infosys as ML Engineer
Education: B.Tech Computer Science, IIT Delhi (2019)
```

The Job Description from the HR team says:

```
We need a Senior Data Scientist with 3+ years experience.
Must know: Python, Deep Learning, TensorFlow or PyTorch.
Good to have: NLP, SQL, data storytelling.
```

■ How the System Works — Step by Step

Step 1 — PDF Ingestion

The system receives Priya's resume as a PDF file. A PDF is not plain text — it is a visual document with fonts, columns, tables, and layouts. The first job is to extract raw text from it.

> A resume has structure. "Python, TensorFlow" appearing under a section titled "Skills" means something very different from the same words appearing in a personal statement. The system is smart enough to understand this layout.

Step 2 — Named Entity Recognition (NER): Reading the Resume Like a Human

Once the text is extracted, the system needs to understand it — not just read it. This is done by a technique called Named Entity Recognition. Think of it as teaching a computer to highlight important things in a document the same way a human recruiter would with a yellow marker.

Step 3 — Hinglish Handling (The Standout Feature)

Many Indian candidates write resumes in a mix of Hindi and English. Standard English-only NLP systems completely fail here. This system includes a Hinglish detection and transliteration module that detects Devanagari or Roman-script Hindi words, transliterates them to English equivalents, and feeds the cleaned text into the main NER pipeline.

> This means a resume written partly in Hindi is just as readable to the system as one written entirely in English — a significant advantage in the Indian HR market.

Step 4 — Sentence-BERT Embeddings: Turning Text Into Math

The system converts Priya's entire professional summary into a list of 768 numbers called a vector or embedding. Two resumes that describe similar experience will have similar numbers. A resume about "machine learning" and a job description asking for "deep learning engineer" will have vectors that are close to each other — because the meanings are related.

> Priya's resume → converted to a 768-number vector. The Job Description → also converted to a 768-number vector. These two vectors look very similar. That means Priya is a strong match.

Step 5 — Cosine Similarity: Calculating the Match Score

Once both resume and job description are vectors, the system calculates how similar they are using cosine similarity. The result is a number between 0 and 1. For Priya, the system returns a score of 0.87 — a strong match. All 400 resumes are scored this way, then sorted from highest to lowest.

Step 6 — Cross-Encoder Reranking (Optional Fine-Tuning)

The cosine similarity pass is fast and gives a rough ranking. But for the top 20 candidates, the system does a second, more careful analysis using a Cross-Encoder model. Where the first pass compares resume and JD independently, the cross-encoder reads them together side by side — catching subtler semantic alignments.

Step 7 — Streamlit Dashboard: The HR Interface

Everything above happens behind the scenes. What the HR manager actually sees is a clean web dashboard. It shows a ranked table of all candidates with match scores, extracted skills displayed as tags, a visual match breakdown showing which JD skills were found or missing, filters by score/experience/skill, and one-click export to Excel.

Match Score Reference

Score Range	Meaning
0.90 – 1.00	Excellent match
0.75 – 0.89	Strong match
0.60 – 0.74	Moderate match
Below 0.60	Weak match

■ Tech Stack — What Each Tool Does

Tool / Library	Role in the System

PyMuPDF / pdfplumber	Extracts raw text and layout from PDF resumes
LayoutLM (Microsoft)	Understands document layout — knows Skills section from body text
spaCy	Named Entity Recognition — labels skills, degrees, companies, dates
LangDetect + IndicNLP	Detects Hindi/Hinglish text and converts it to English
Sentence-BERT (SBERT)	Converts resume and JD text into 768-dimensional semantic vectors
scikit-learn	Calculates cosine similarity between resume and JD vectors
Cross-Encoder (HuggingFace)	Fine-grained reranking of top candidates
Streamlit	Builds the web dashboard HR teams interact with
pandas	Stores and manipulates candidate data for display and export
FastAPI (optional)	Wraps the backend as an API for HR software integration

■ What the Output Looks Like

After processing all 400 resumes against the Senior Data Scientist JD, the HR manager sees:

Rank	Candidate	Score	Key Skills Found	Experience
1	Priya Sharma	0.91	Python, TensorFlow, SQL, ML	4 years
2	Rahul Mehta	0.88	Python, PyTorch, NLP, DL	5 years
3	Anita Joshi	0.82	Python, TensorFlow, Viz	3 years
...	...	...	...	...
400	Vikas Thakur	0.31	Excel, PowerPoint	1 year

■ What Makes This Project Stand Out

1. Hinglish Resume Support

No mainstream ATS (Applicant Tracking System) handles Hinglish. This system does. In India, where a large share of the workforce writes semi-formal Hinglish in resumes, this is a real competitive advantage.

2. Layout Awareness

Many resume parsers extract text linearly and lose structure. Using LayoutLM, this system understands that text positioned after a "Skills" heading is a skill — not just a word.

3. Semantic Matching, Not Keyword Matching

Traditional ATS tools count how many keywords from the JD appear in the resume. This system understands meaning. A resume that says "built predictive models using scikit-learn" will match a JD asking for "machine learning experience" — even though none of those exact words appear in the other document.

4. Explainability

The dashboard shows why a candidate ranked highly — which skills matched, which were missing. This helps HR teams make defensible decisions, not just take a number on faith.

■ Who Is This For?

User	How They Benefit
HR Managers	Shortlist 400 resumes in minutes instead of days
Recruiters	Surface hidden talent who wrote good resumes in non-standard formats
Startups	Get enterprise-grade recruitment intelligence without expensive ATS software
HR Tech Builders	A modular codebase to build a SaaS product on top of

■ Future Enhancements

• Multi-JD matching: Rank one candidate against multiple open roles simultaneously

• Bias detection module: Flag if ranking correlates with gender names or college tiers

• Interview question generator: Auto-generate questions for shortlisted candidates based on skill gaps

• ATS integration: Plug into Zoho Recruit, Keka, or Darwinbox via API

• Voice resume support: Accept audio resumes and transcribe before processing

---

Built with spaCy · Sentence-Transformers · LayoutLM · Streamlit · Love for the Indian HR market